

Universidad Nacional Autónoma de México

Facultad de Ingeniería

Sistemas Distribuidos

Mexi Banco: saga orquestada y circuit breaker

Bitácora del laboratorio S08

Alumno: Christian Franco
Cuenta: 117003136
Materia: Sistemas Distribuidos
Grupo: Grupo 2
Fecha: 17 de octubre de 2026

Introducción

En el laboratorio anterior separamos Mexi Banco en cinco servicios con sus propias bases de datos. La separación resolvió el despliegue independiente, pero dejó un hueco: la transferencia interna ya no es atómica. El método `transferir()` hace cuatro saltos HTTP y cada uno se confirma por su cuenta. Si el abono falla después del cargo, el dinero sale de origen y no llega a destino.

Esta práctica sustituye esa llamada directa por una **saga orquestada** y protege la comunicación con un **circuit breaker** manual. Al final: la transferencia tiene un estado persistido en cada paso, sabe compensar cuando algo falla después del cargo, deja un estado explícito cuando la compensación tampoco es posible, y responde rápido cuando `cuenta` está caída.

Entorno. Todo se ejecutó con Docker Engine y Docker Compose v2 sobre Ubuntu, en la rama `03-gateway-discovery-balanceo` del repositorio `mexi-banco`.

1. El problema, reproducido

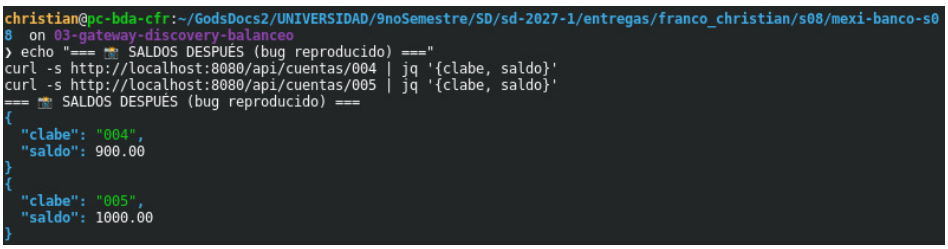
Antes de tocar código se reprodujo el bug con la variable `PAUSA_ENTRE_CARGO_Y_ABONO_MS=20000`. El método `pausaDeDemo()` duerme 20 segundos entre el cargo y el abono, tiempo suficiente para apagar `cuenta` a mano.

Procedimiento:

1. Saldos iniciales de dos cuentas (1000 y 1000).
2. Levantar transferencia con `PAUSA_ENTRE_CARGO_Y_ABONO_MS=20000 docker compose up -d transferencia`.
3. Lanzar una transferencia de 100 y, a los 5 segundos, ejecutar `docker compose stop cuenta`.
4. Al terminar la pausa, el abono lanza `IllegalStateException: No instances available for cuenta`. Nadie deshace el cargo.
5. Encender `cuenta` y verificar saldos.

Cuenta	Saldo antes	Saldo después	Diferencia
`004` (origen)	1000	**900**	−100
`005` (destino)	1000	1000	0
Total del sistema	2000	**1900**	−100

El motivo, en una frase: cada llamada HTTP a `cuenta` confirma por su cuenta y no hay transacción que abarque al cargo y al abono. Si el segundo falla, el primero ya es visible para siempre.



2. Diseño de la saga

2.1 Estados de la transferencia

Modelé una saga orquestada donde `TransferenciaService` es el orquestador. Cada paso actualiza el estado en su propia transacción corta, de modo que si el servicio se cae a la mitad, la base diga en qué paso quedó. Los estados son:

Estado	Significado
`INICIADA`	Se guardó la transferencia pero aún no se movió dinero.
`CARGADA`	El cargo a origen ya se aplicó; el abono aún no.
`ABONADA`	El abono a destino también; los movimientos aún no.
`COMPLETADA`	Todo terminó correctamente.
`FALLIDA`	Falló antes del cargo; no hay nada que compensar.
`COMPENSADA`	Falló después del cargo y la compensación funcionó.
`COMPENSACION_PENDIENTE`	Falló después del cargo y la compensación también falló.

2.2 Pasos y compensación

Paso	Acción (transacción local)	Compensación
T1	Cargar `claveOrigen` en `cuenta`	Abonar el mismo monto a `claveOrigen` con `Idempotency-Key`.
T2	Abonar `claveDestino`	— (no hay nada que deshacer si T2 fue lo que falló).
T3	Registrar dos asientos en `movimiento`	— (best-effort; no bloquea la saga).
T4	Notificar al cliente	— (irreversible; va al final a propósito).

Si T2 falla y T1 ya pasó, se compensa T1 (reembolso a origen). Si la compensación también falla, la transferencia se queda en `COMPENSACION_PENDIENTE`. Ese estado es la pista que un trabajo programado usaría después para terminar la operación.

3. El código clave

3.1 El orquestador

El método `transferir()` persiste el estado en cada paso y, en `catch`, delega a `compensar()` cuando corresponde.

```
public Transferencia transferir(String claveOrigen,
                                String claveDestino,
                                BigDecimal monto) {
    Transferencia t = transferencias.save(
            new Transferencia(claveOrigen, claveDestino, monto));
    log.info("[SAGA {}] INICIADA", t.getId());

    try {
```

```
        CuentaRemota origen = breaker.ejecutar(
            () -> cuentas.cargar(claveOrigen, monto, t.getId() + ":cargo"));
        t.setEstado(EstadoTransferencia.CARGADA);
        t = transferencias.save(t);

        pausaDeDemo();

        if (falloAbonoClabe != null && falloAbonoClabe.equals(claveDestino))
            throw new ReglaDeNegocioException("Fallo FORZADO para " +
                claveDestino);

        CuentaRemota destino = breaker.ejecutar(
            () -> cuentas.abonar(claveDestino, monto, t.getId() + ":abono"));
        t.setEstado(EstadoTransferencia.ABONADA);
        t = transferencias.save(t);

        /* T3: movimientos + notificaciones (best-effort) */
        t.setEstado(EstadoTransferencia.COMPLETADA);
        return transferencias.save(t);

    } catch (RuntimeException falla) {
        log.error("[SAGA {}] fallo en estado {}: {}",
                  t.getId(), t.getEstado(), falla.getMessage());
        return compensar(t, falla);
    }
}
```

3.2 La compensación

Si la transferencia ya estaba en `CARGADA`, se reembolsa al origen con una llave idempotente. Si el reembolso falla, se marca `COMPENSACION_PENDIENTE`.

```
private Transferencia compensar(Transferencia t, RuntimeException causa) {
    if (t.getEstado() != EstadoTransferencia.CARGADA) {
        t.setEstado(EstadoTransferencia.FALLIDA);
        t.setMotivoFallo(causa.getMessage());
        return transferencias.save(t);
    }
    try {
        breaker.ejecutar(() -> cuentas.abonar(
            t.getClaveOrigen(), t.getMonto(),
            t.getId() + ":compensacion"));
        t.setEstado(EstadoTransferencia.COMPENSADA);
        t.setMotivoFallo("Compensada tras: " + causa.getMessage());
    } catch (RuntimeException fallaComp) {
        t.setEstado(EstadoTransferencia.COMPENSACION_PENDIENTE);
        t.setMotivoFallo("Compensacion pendiente: " + fallaComp.getMessage());
    }
    return transferencias.save(t);
}
```

La llave `<id>:compensacion` viaja como cabecera `Idempotency-Key`. Si el reintento llega después de un timeout, el servicio `cuenta` debe reconocer la llave y **no duplicar el abono**. Sin eso, reintentar cobra dos veces.

3.3 El circuit breaker manual

Implementé el breaker a mano (la variante que suma puntos). Los tres estados viven en un solo método `ejecutar`.

```
public synchronized <T> T ejecutar(Supplier<T> llamada) {
    if (estado == Estado.ABIERTO) {
        if (Instant.now().isBefore(abiertoDesde.plus(espera)))
            throw new CircuitBreakerAbiertoException(
                "Circuit breaker abierto: reintento en ...");
        estado = Estado.MEDIO_ABIERTO;
    }
    try {
        T r = llamada.get();
        estado = Estado.CERRADO;
        fallasConsecutivas = 0;
        return r;
    } catch (RuntimeException falla) {
        if (estado == Estado.MEDIO_ABIERTO) {
            estado = Estado.ABIERTO; abiertoDesde = Instant.now();
        } else if (++fallasConsecutivas >= umbral) {
            estado = Estado.ABIERTO; abiertoDesde = Instant.now();
        }
        throw falla;
    }
}
```

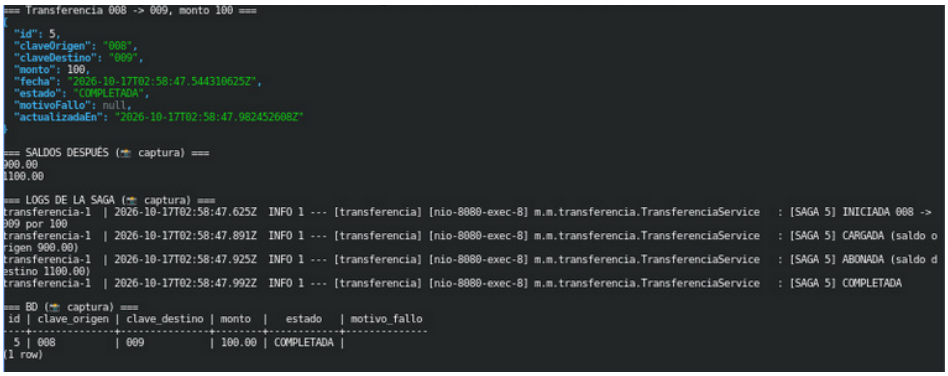
4. Compensación funcionando

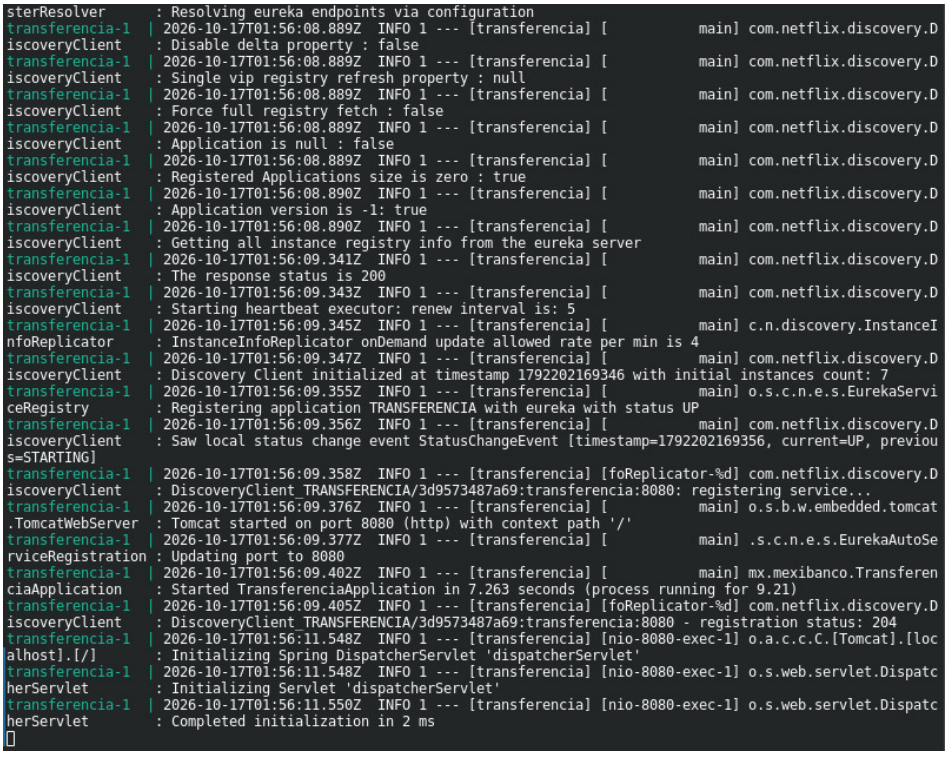
Se agregó una variable de entorno `FALLO_ABONO_CLABE` para forzar el fallo del abono hacia una CLABE específica, sin apagar `cuenta`. Es útil para probar la compensación sin simular una caída completa.

Procedimiento: reiniciar transferencia con `FALLO_ABONO_CLABE=009`, lanzar la transferencia de 100 de `008` a `009`.

Cuenta	Antes	Después
`008` (origen)	700	**700** (restaurado por compensación)
`009` (destino)	1300	1300 (nunca se abonó)

La transferencia queda en `COMPENSADA` con `motivoFallo = Çompensada tras: Fallo de abono FORZADO para 009"`. Los logs muestran `CARGADA` → `fallo` → `COMPENSANDO` → `COMPENSADA`.





5. Cuando la compensación también falla

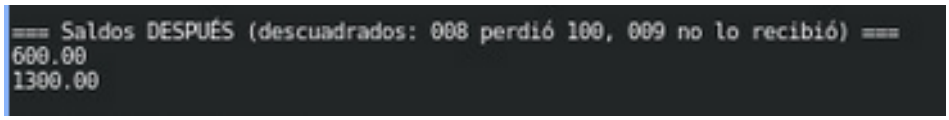
Se repitió la Parte 0 con la saga ya instalada: pausa de 20 segundos, apagar `cuenta` a los 5 segundos, dejar que el abono falle y que la compensación también falle (porque `cuenta` sigue apagada). La fila en la base quedó así:

```
id=9   estado=COMPENSACION_PENDIENTE
motivo_fallo = Çompensacion pendiente: I/O error on POST request for
        http://cuenta/cuentas/008/abonos: Connect timed out"
```

Los saldos quedaron descuadrados: `008=600`, `009=1300`. El dinero está en el limbo, pero el sistema **lo sabe**: la fila existe con un estado que implica que hay que devolver 100 a `008`. Un trabajo programado con `@Scheduled(fixedDelay = 30_000)` que consulte las filas en `COMPENSACION_PENDIENTE` y las

reintente resolvería la situación. Por eso la **compensación debe ser idempotente**: un reintento tras un timeout no puede cobrar dos veces.





6. Circuit breaker

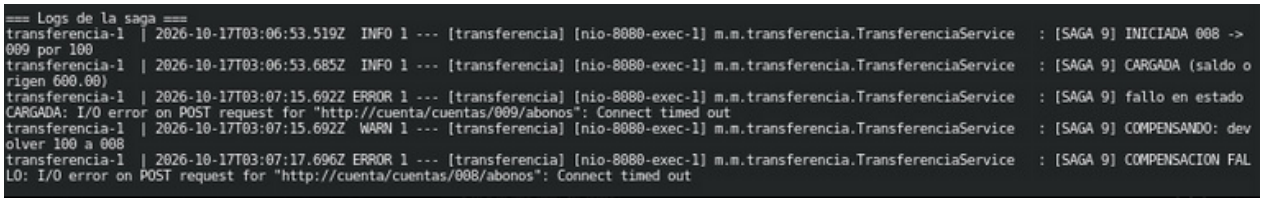
Se apagó `cuenta`, se lanzaron varias transferencias seguidas y se observaron los tres estados en el endpoint interno `/diagnostico/circuit-breaker` y en los logs del servicio.

Estado	Cómo se alcanzó	Comportamiento
CERRADO	Estado inicial	Pasan todas las llamadas; cuenta fallas seguidas.
ABIERTO	3 fallas consecutivas (umbral)	Rechazo rápido, sin llamar a `cuenta` (< 50 ms).
MEDIO_ABIERTO	Espera de 15 s agotada	Deja pasar 1 llamada de prueba.
CERRADO	La prueba salió bien	Reinicia `fallas=0`.

Evidencia de un ciclo completo en una sola corrida:

```
[CB] falla 1/3 -> No instances available for cuenta
[CB] falla 2/3 -> No instances available for cuenta
[CB] falla 3/3 -> No instances available for cuenta
[CB] UMBRAL alcanzado -> ABIERTO por 15s
[CB] ABIERTO -> rechazo rapido (reintento en 14s)
[CB] ABIERTO -> rechazo rapido (reintento en 13s)
[CB] -> MEDIO_ABIERTO: dejo pasar 1 llamada de prueba
[CB] prueba OK -> CERRADO (fallas=0)
```

Con `cuenta` caída, los primeros intentos tardan cerca de 1–2 segundos (timeout HTTP); a partir del umbral, la respuesta llega en milisegundos porque el breaker rechaza sin salir a la red.





7. Lo que la saga no te da

Una saga sustituye la atomicidad de una transacción local por **atomicidad eventual con compensaciones**; no reemplaza el aislamiento. Entre el cargo y la compensación de la transferencia `id=9`, cualquier consulta al saldo de `008` vio 600 pesos aunque el sistema ya sabía que había una devolución pendiente. Ese estado intermedio es inevitable cuando cada servicio tiene su propia base, y se mitiga con estados explícitos (`PENDIENTE`), no con candados.

Para acercarse al aislamiento real harían falta 2PC sobre las bases (con el costo de bloquear a todos los participantes) o un patrón con reservas temporales. Ambos quedan fuera del alcance del laboratorio.

Conclusión

Este laboratorio mostró dos piezas nuevas que se apoyan entre sí. La saga resuelve la atomicidad perdida al partir el monolito: cada paso tiene su compensación, y la fila de la transferencia guarda en qué punto está, de modo que un fallo intermedio no borra la evidencia. El breaker complementa la saga evitando que un servicio lento tumbe a los demás: cuando `cuenta` deja de responder, transferencia rechaza rápido en lugar de acumular peticiones abiertas.

La parte menos intuitiva fue comprobar que la compensación puede fallar por la misma razón que el abono original, y que la respuesta correcta no es reintentar sin control sino marcar la transferencia como `COMPENSACION_PENDIENTE` y dejar que un proceso posterior la termine con idempotencia.